Penerapan MA-Stat-DSM pada Pertandingan Basket

Vince Farrel Natanael–6182301024

1 Data Skripsi

Pembimbing utama/tunggal: **Lionov, Ph.D.**
Pembimbing pendamping: -
Kode Topik : **LNV6105**
Topik ini sudah dikerjakan selama : **1** semester
Pengambilan pertama kali topik ini pada : Semester **61 - Ganjil 26/27**
Pengambilan pertama kali topik ini di kuliah : **Tugas Akhir 1**
Tipe Laporan : **B -** Dokumen untuk reviewer pada presentasi dan **review Skripsi 1**

2 Latar Belakang

Stat-DSM merupakan sebuah metode untuk memisahkan dua kelas lintasan dimana setiap sub-lintasan pada setiap kelas memiliki perbedaan yang cukup sehingga dapat dinyatakan bahwa perbedaan tersebut merupakan signifikan secara statistik dan tidak terjadi hanyak karena acak. Dalam stat-DSM, anggap terdapat $N = 1000$ lintasan. Setiap lintasan memiliki kelas atau label $Y_i \in \{0, 1\}$ dimana $Y_i = 1$ menandakan sukses dan $Y_i = 0$ menandakan tidak sukses. Setiap lintasan T_i terdiri dari beberapa titik-titik lokasi yang dinotasikan dengan $T_i = (p_i^1, p_i^2, ..., p_i^n)$ dimana setiap $p_i^t = (x_i^t, y_i^t)$ yang merupakan koordinat x dan y.

Sub-lintasan merupakan sebuah bagian kecil dari sebuah lintasan yang bersifat kontinuous. Anggap kita memiliki sebuah lintasan $T_i = (p_i^1, p_i^2, p_i^3, p_i^4, p_i^5, p_i^6, p_i^7, p_i^8)$, sebuah sub-lintasan dapat berupa $S_i = (p_i^3, p_i^4, p_i^5, p_i^6, p_i^7)$

Stat-DSM melakukan analisis terhadapa sub-lintasan, alih-alih melakuka analisis terhadap seluruh lintasan karena sebagian besar dari sebuah lintasan diisi dengan informasi yang tidak relevan. Sehingga apa yang algoritma tersebut ingin analisis hanya bagian-bagian yang dianggap penting saja.

Pada intinya, algoritma Stat-DSM ingin menanyakan dua hal yaitu

1. Apakah lintasan T_i mengandung pola-pola sub-lintasan yang cukup mirip dengan S?
2. Apakah pola sub-lintasan S tersebut secara statistikal terjadi secara berbeda di antara kedua kelas tersebut?

Kedua permasalahan tersebut seputar dua hal berbeda, yang pertama merupakan permasalahan mengenai geometri dan yang kedua mengenai statistik.

Untuk permasalahan yang pertama, Stat-DSM perlu dapat mencari kemiripan antar lintasan secara geometri. Stat-DSM akan mencari pola-pola pergerakan dalam sebuah lintasan yang sama dengan melihat titik-titik pada sebuah lintasan. Tetapi tidak dapat sebatas dilakukannya perbandingan dan melihat apakah sama atau tidak karena setiap titik demi titik tidak akan identikal. Oleh karen itu diperlukannya sebuah fungsi jarak untuk menyatakan seberapa mirip titik-titik kedua lintasan untuk dianggap sebagai sama. Fungsi jarak tersebut secara umum dinotasikan dengan $d(S, T) \leq \epsilon$ dimana d = fungsi jarak dan ϵ = similarity threshold. Anggap terdapat sebuah kandidat sub-lintasan S, untuk setiap lintasan akan dilihat

$$X_i(S) = \begin{cases} 1 & \text{jika } S \text{ ditemukan dalam } T_i \\ 0 & \text{selainnya} \end{cases} \tag{1}$$

sehingga kita akan memiliki contoh dataset

Lintasan	**Label**	**Memiliki** S
T1	Positif	1
T2	Positif	1
T3	Positif	0
T4	Positif	1
T5	Negatif	0
T6	Negatif	0
T7	Negatif	1
T8	Negatif	0

Sekarang disisakan dengan $X_i \in \{0, 1\}$ dan $Y_i \in \{0, 1\}$ dimana dapat digunakan untuk menyelesaikan permasalah kedua.

Dengan adanya tabel tersebut, permasalah kedua dapat mulai diselesaikan dengan menjawab apakah pola sub-lintasan S yang muncul dalam berbagai lintasan terjadi berdasarkan label yang ada. Dengan ini dinyatakannya null hipotesis $H_0 : X \perp Y$ yang bermakna bahwa pola kemunculan sub-lintasan S dalam sebuah lintasan bersifat independen dengan label kelas lintasan tersebut dan $H_1 : X \not\perp Y$ yang bermakna bahwa pola kemunculan sub-lintasan S dalam sebuah lintasan dan label kelas sebuah lintasan memiliki hubungan.

Selanjutnya diperlukannya dibentuk sebuah *contingency table*. Anggap kita menemukan untuk label kelas positif 80 mengandung S dan 20 tidak dan untuk label kelas negatif 30 mengandung S dan 70 tidak. Sehingga didapatkannya

Tabel 1: A 2×2 Contingency Table

	Mengandung		
Kelas	S	$\neg S$	**Total**
Positif	80	20	100
Negatif	30	70	100
Total	110	90	200

Setelah didapatnya *contingency table* tersebut, Stat-DSM menggunakan *Fisher's Exact Test* untuk melihat hubungan antara adanya S dengan label kelas. untuk tabel

Label	S	$\neg S$
Positif	a	b
Negatif	c	d

Probabilitas untuk memperoleh tabel yang diamati di bawah hipotesis nol didasarkan pada distribusi hipergeometrik atau kemungkinan mendapatkan data tersebut (atau yang lebih ekstrem) murni karena kebetulan adalah

$$P = \frac{\binom{a+b}{a}\binom{c+d}{c}}{\binom{a+b+c+d}{a+c}}$$

dan hasil dari persamaan tersebut adalah p-value

Apa yang pengertian dari p-value? Anggap terdapat $p = 0.003$, maka jika hipotesis null adalah *true*, maka kemungkinan mendapatkannya hasil yang sama atau lebih extrem memiliki kemungkinan 0.3%. Sehingga jika p-value kecil memberikan bukti untuk dapat menolak hipotesis H_0 yang pada kasus ini berarti bahwa label kelas dan kemunculan S dalam sebuah lintasan memiliki hubungan dan tidak muncul hanya karena peluang.

Tetapi perhitungan ini hanya berlaku untuk klasifikasi statistikal yang dilakukan sekali saja. Anggap terdapat 100,000 kandidat sub-lintasan, maka akan dilakukannya 100,000 *Fisher tests*. Saat dilakukannya banyak tes, maka kemungkinan ditemukannya hasil yang sama juga akan naik karena kemungkinannya berubah dari menemukan sebuah hasil yang sama menjadi kemungkinan minimal satu hasil yang sama ditemukan. Masalah ini sebagai *Family-Wise Error Rate* yang dapat dihitung sebagai $FWER = 1-(1-\alpha)^m$. Contohnya adalah dengan $\alpha = 0.05$ dengan melakukan 20 tes, maka kemungkinan adanya kemunculan naik menjadi 64% Inilah mengapa dibutuhkannya sebuah metode untuk mengkoreksi jika dilakukannya beberapa tes. Untuk Stat-DSM, digunakannya metode yang didasarkan dengan *Westfall-Young Permutation*.

Bagaimana metode *Westfall-Young Permutation* bekerja? Pertama metode tersebut melakukan permutasi terhadap dataset yang ada. Anggap terdapat

Lintasan	**Pola**	**Label**
T1	1	Positif
T2	1	Positif
T3	0	Negatif
T4	0	Negatif

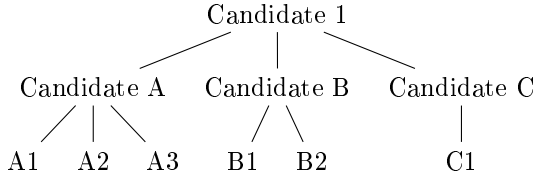
Permutasi akan dilakukan terhadap label tersebut, sehingga algoritma akan mengubah tabel tersebut menjadi

Lintasan	**Pola**	**Label**
T1	1	Negatif
T2	1	Positif
T3	0	Positif
T4	0	Negatif

Dengan melakukan ini, relasi antara X dengan Y dapat diubah tanpa mengubah struktur data sehingga korelasi antar data masih di preservasi. Setelah label diacak, akan dihitung lagi p-value dari berbagai permutasi yang dilakukan. Setelah beberapa permutasi, kita akan mendapatkan beberapa p-value dengan nilai yang berbeda-beda. Pada setiap permutasi, akan diambil hasil p-value terkecil.

Salah satu hasil buruk dari menggunakan metode ini adalah seberapa besarnya ruang pencariannya. Oleh karena itu dibutuhkannya digunakannya beberapa metode untuk membuat program lebih effisien. Salah satu metodenya adalah *minimum trajectory length* (l_min) yang bekerja untuk membatasi seberapa kecil panjang sub-lintasan yang dapat diproses. Contohnya adalah jika $l_min = 5$, maka sub-lintasan yang akan diproses hanya akan memiliki panjang 5 atau lebih.

Salah satu metode lainnya adalah dengan menggunakan sebuah *candidate tree*. Candidate tree merupakan pohon yang menampung list sub-lintasan, dimana pada pohon tersebut, kandidat dengan panjang yang lebih besar memiliki relasi dengan kandidat yang lebih kecil. Contoh *candidate tree*



Gambar 1: Contoh *Candidate Tree*

Lalu setelah kita memiliki *Candidate tree*, kami dapat mengimplementasikan sebuah pruning startegy dimana jika kita dapat mengetahui sebuah branch tidak dapat menghasilkan sebuah pola yang diskriminan secara statistik, kami dapat prune branch tersebut.

Secara menyeluruh, cara kerja stat-DSM adalah dengan

1. Generate kandidat sub-lintasan S
2. Mencari dataset untuk lintasan-lintasan yang memiliki segmen dengan jarak yang mirip dengan S dinotasikan dengan $d(S, T_i) \leq \epsilon$
3. Lalu representasi setiap lintasan dengan $X_i(S) \in \{0, 1\}$
4. Gabungkan X_i yang baru dibuat dengan label kelas Y_i
5. Buatlah *contingency table*
6. Lakukannya *Fisher's Exact Test* untuk mendapatkan p-value
7. Lakukan *Westfall-Young Permutation Correction* untuk mendapatkan *significant threshold* yang dikoreksi
8. Tentukan apakah branch perlu di explore lagi, jika ya ulangi, jika tidak prune

Contoh tes kasus untuk pengerjaan Stat-DSM :
Parameter :

1. $n = 10$
2. $n_+ = 5$, $n_- = 5$
3. $min_l = 2$
4. $\epsilon = 0$
5. $\alpha = 0.05$

Data :

Lintasan	**Label** Y_i	**Alur Koordinat**
T_1	1	$(0,0) \to (1,0) \to (2,0) \to (3,0)$
T_2	1	$(0,0) \to (1,0) \to (2,0) \to (3,0)$
T_3	1	$(0,0) \to (1,0) \to (2,0) \to (3,0)$
T_4	1	$(0,0) \to (1,0) \to (2,0) \to (3,1)$
T_5	1	$(0,0) \to (1,0) \to (2,1) \to (3,1)$
T_6	0	$(0,5) \to (1,5) \to (2,5) \to (3,5)$
T_7	0	$(0,10) \to (1,10) \to (2,10) \to (3,10)$
T_8	0	$(0,15) \to (1,15) \to (2,15) \to (3,15)$
T_9	0	$(0,20) \to (1,20) \to (2,20) \to (3,20)$
T_{10}	0	$(0,25) \to (1,25) \to (2,25) \to (3,25)$

1. **Generate kandidat sub-lintasan** S
   Pertama harus mengambil sub-lintasan dari data set yang panjang sub-lintasan tidak kurang dari $min_l = 2$
   Contoh :
   $S_a = [(0,0), (1,0)]$
   $S_b = [(0,0), (1,0), (2,0)]$
   $S_c = [(0,0), (1,0), (2,0), (3,0)]$

2. **Mencari lintasan yang memenuhi** $d(S, T_i) \leq \epsilon$
   Untuk setiap kandidat yang ditemukan, algoritma kemudian akan mencari lintasan yang memiliki segmen sub-lintasan yang sesuai dengan kandidat tersebut. Pada tes kasus ini, ditetapkan $\epsilon = 0$ yang berarti bahwa agar dinyatakan dua segmen sebagai mirip, jaraknya harus ≤ 0.

Contoh :

Kandidat	**Lintasan yang cocok**	**Support**
$S_a = [(0,0),(1,0)]$	T_1, T_2, T_3, T_4, T_5	5
$S_b = [(1,0),(2,0)]$	T_1, T_2, T_3, T_4	4
$S_c = [(1,0),(2,1)]$	T_5	1
$S_d = [(0,0),(1,0),(2,0)]$	T_1, T_2, T_3, T_4	4
$S_e = [(0,0),(1,0),(2,1)]$	T_5	1
$S_f = [(0,0),(1,0),(2,0),(3,0)]$	T_1, T_2, T_3	3

3. **Merepresentasikan setiap lintasan dengan** $X_i(S) \in \{0,1\}$

   Representasikan dengan

$$X_i(S) = \begin{cases} 1 & d(S,T_i) \leq \epsilon \\ 0 & d(S,T_i) \geq \epsilon \end{cases} \tag{2}$$

   Sehingga

   $X(S_a) = (1,1,1,1,1,0,0,0,0,0)$

   $X(S_b) = (1,1,1,1,0,0,0,0,0,0)$

   $X(S_c) = (1,1,1,0,0,0,0,0,0,0)$

4. **Menggabungkan** $X_i(S)$ **dengan label kelas** Y_i

   Jadi untuk lintasan $X(S_a)$ didapat

Lintasan	**Label** Y_i	**Contain** S_a**?**
T_1	1	1
T_2	1	1
T_3	1	1
T_4	1	1
T_5	1	1
T_6	0	0
T_7	0	0
T_8	0	0
T_9	0	0
T_{10}	0	0

5. **Membentuk contingency table**

Tabel 2: A 2×2 Contingency Table of $X(S_a)$

	Mengandung		
Kelas	S_a	$\neg S_a$	**Total**
$Y = 1$	5	0	5
$Y = 0$	0	5	5
Total	5	5	10

6. **Melakukan Fisher's Exact Test untuk mendapatkan** p**-value**

   Fisher's Exact Test disini digunakan untuk menguji hubungan antara keberadaan kandidat sub-lintasan dan label kelas.

Untuk kasus S_a, p-value dapat dihitung dengan pertama mencari

$$P_upper = \frac{\binom{5}{5}\binom{5}{0}}{\binom{10}{5}} = \frac{1}{252}$$

Maka untuk uji sua sisi, untuk mendapatkan peluang kedua sisi, perlu diperhatikan juga peluang P_lower dengan

$$\begin{aligned} p(S_A) &= 2\left(\frac{1}{252}\right) \\ &= \frac{2}{252} \\ &\approx 0.00794. \end{aligned}$$

7. **Melakukan Westfall-Young Permutation Correction** Pada tahap ini, anggap telah terdapat banyak kandidat yang diperiksa. Oleh karena itu, kemungkinan adanya kandidat yang menghasilkan p-value kecil secara kebetulan akan meningkat. Untuk tahap ini, kita akan melakukan permutasi terhadap label Y. Anggap kita melakukan permutasi label Y dan membuat label baru $Y^1 = (1, 1, 1, 0, 0, 1, 0, 0, 1, 0)$. Kemudian akan dilakukannya lagi Fisher's Exact Test terhadap semua kandidat tadi tetapi menggunakan label Y^1. Karena kita memiliki 10 lintasan dengan 5 positif dan 5 negatif sehingga jumlah permutasi yang dilakukan yaitu

$$B = \binom{10}{5} = \frac{10!}{5! * 5!} = 252$$

kombinasi berbeda. Pada setiap batch permutasi akan dibentuk list p-value baru, dari semua p-value yang ditemukan, ambil yang terkecil.

Contohnya adalah untuk $Y^1 = (1, 1, 1, 0, 0, 1, 0, 0, 1, 0)$
Anggap kita dapat

Kandidat	p-**value**
S_a	0.16667
S_b	0.52381
S_c	0.52381

Maka kita akan mengambil $P^1_{min} = 0.16667$.

Contoh kita akan mencoba melakukan beberapa permutasi

Permutasi	$p(S_a)$	$p(S_b)$	$p(S_c)$	**Minimum**
Original	0.00794	0.04762	0.16667	0.00794
1	0.16667	0.52381	0.52381	0.16667
2	0.04762	0.16667	0.52381	0.04762
3	0.52381	0.04762	0.16667	0.04762
4	0.00794	0.04762	0.16667	0.00794
5	0.16667	1.00000	0.52381	0.16667
...	...	...	...	...

Dan didapatkan distribusi

$$0.16667, 0.04762, 0.04762, 0.00794, 0.16667, \ldots$$

Selanjutnya untuk mendapatkan threshold yang baru, setelah melakukan contohnya 252 permutasi, kami akan mempunyai 252 p^x_{min}. Nilai tersebut kemudian akan diurutkan mulai dari yang paling kecil hingga yang paling besar. Contohnya :

Urutan	Minimum p-value
1	0.00794
2	0.00794
3	0.04762
4	0.04762
...	...
12	0.04762
13	0.05
...	...
252	0.05

Sekarang dapat dengan $\alpha = 0.05$ dan $B = 252$ dapat dihitung sebagai

$$\lfloor \alpha B = 0.05(252) = 12.6 \rfloor$$

Sehingga dari tabel list p_min^x kami mengambil nilai ke-12 yaitu $\delta = 0.04762$ dan inilah yang akan menjadi threshold baru. Kita kemudian akan melihat lagi nilai-nilai p-value yang menggunakan data label original dan filter lagi menggunakan threshold yang baru.

8. Menentukan apakah cabang perlu dieksplorasi atau di-prune
   Sekarang setelah kita mendapatkan threshold baru, kita dapat menentukan mana yang dapat di *prune*. Pada stat-dsm, algoritma menentukan bagian mana yang dapat di *prune* menggunakan lower bound terhadap p-value untuk menentukan apa yang dapat dicapai oleh descendant dari suatu node. Stat-DSM kemudian membandingkan lower bound tersebut dengan threshold yang berasal dari distribusi Westfall-Young. Pruning dan pembentukan distribusi WY dilakukan secara bersamaan untuk mengurangi jumlah kandidat yang harus diproses. Anggap kita memiliki $\delta = 0.02$, kemudian kita menghitung lower bound S_d yaitu $LB(S_d) = 0.05$. Angka ini memiliki arti bahwa semua kandidat descendant dari $S_d \geq 0.05$ dan karena $\delta = 0.02$ sehingga $0.05 \geq 0.02$ dan algoritma tahu untuk melakukan *prune* terhadap S_d Semakin kecil support pada suatu sub-lintasan makan semakin sulit juga untuk kandidat tersebut untuk menghasilkan p-value yang kecil. Setalah dilakukannya *pruning*, dibuatnya lagi kandidat-kandidat baru dengan length yang lebih panjang dari branch yang berhasil lolos dan dilakukannya lagi semua langkah dari awal.

3 Rumusan Masalah

4 Tujuan

5 Detail Perkembangan Pengerjaan Skripsi

Detail bagian pekerjaan skripsi sesuai dengan rencan kerja/laporan perkembangan terkahir :

1. **Melakukan survei ke Museum Geologi Bandung untuk mendapatkan denah serta mengetahui perilaku pengunjung museum secara umum (arah perjalanan, kecepatan, lama melihat objek, dll).**
   Status : Ada sejak rencana kerja skripsi.
   Hasil : Survei sudah dilakukan sebanyak 3x pada tanggal X, Y dan Z. Pada kunjungan pertama, diperhaitkan denah museum dan dibuat sketsa berdasarkan pengamatan. Pada kunjunga kedua, bertemu dengan bagian humas museum dan berhasil mendapatkan denah serta melakukan wawancara. Hasil wawancara sudah dibuat dan ada softcopy-nya. Pada kunjungan ketiga, secara khusus dilihat

perilaku pengunjung. Masih direncakanan 2x kunjungan lagi. Bukti-bukti kunjungan dapat dilihat di lampiran

2. **Melakukan analisis pada hasil survei terhadap pergerakan pengunjung di museum dan membuat rancangan denah di komputer yang dilengkapi dengan penghalang dan objek di museum.**
   Status : Ada sejak rencana kerja skripsi.
   Hasil :

3. **Melakukan studi literatur mengenai sifat kolektif suatu kerumunan, teknik** *social force model* **dan teknik** *flow tiles*
   Status : Ada sejak rencana kerja skripsi.
   Hasil :

4. **Mempelajari bahasa pemrograman C++ dan cara menggunakan framework OpenSteer**
   Status : Ada sejak rencana kerja skripsi.
   Hasil :

5. **Merancang pergerakan kerumunan di dalam museum menggunakan teknik** *social force model* **dan** *flow tiles* **serta menggunakan teknik lainnya seperti konsep pathway dan waypoints. Selain itu, dirancang pula adanya waktu tunggu (pada saat pengunjung melihat objek di museum) dan cara pembuatan jalur bagi setiap individu pengunjung**
   Status : Ada sejak rencana kerja skripsi.
   Hasil :

6. **Melakukan analisa dan merancang struktur data yang cocok untuk menyimpan penghalang (obstacle)**
   Status : dihapuskan/tidak dikerjakan
   Hasil : berdasarkan analisis singkat, tidak dilakukan analisis lebih jauh karena tidak diperlukan struktur data baru, karena sudah disediakan oleh OpenSteer versi terbaru

7. **Mengimplementasikan keseluruhan algoritma dan struktur data yang dirancang, dengan menggunakan framework OpenSteer**
   Status : Ada sejak rencana kerja skripsi.
   Hasil :

8. **Melakukan pengujian (dan eksperimen) yang melibatkan responde untuk menilai hasil simulasi secara kualitatif**
   Status : Ada sejak rencana kerja skripsi.
   Hasil :

9. **Menulis dokumen skripsi**
   Status : Ada sejak rencana kerja skripsi.
   Hasil : Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

10. **Mempelajari cara menggunakan fitur manipulasi obstacle yang disediakan oleh framework Opensteer versi terbaru**
    Status : baru ditambahkan pada semester ini
    Hasil : baru direncanakan karena framework Opensteer versi paling akhir baru selesai diinstall dan dilihat-lihat bagian contoh-contoh simulasinya

6 Pencapaian Rencana Kerja

Langkah-langkah kerja yang berhasil diselesaikan dalam Skripsi 1 ini adalah sebagai berikut:

1.
2.
3.

7 Kendala yang Dihadapi

Kendala - kendala yang dihadapi selama mengerjakan skripsi :

- Terlalu banyak melakukan prokratinasi
- Terlalu banyak godaan berupa hiburan (game, film, dll)
- Skripsi diambil bersamaan dengan kuliah ASD karena selama 5 semester pertama kuliah tersebut sangat dihindari dan tidak diambil, dan selama 4 semester terakhir kuliah tersebut selalu mendapat nilai E
- Mengalami kesulitan pada saat sudah mulai membuat program komputer karena selama ini selalu dibantu teman

Bandung, 01/01/1900

Vince Farrel Natanael

Menyetujui,

Nama: Lionov, Ph.D.
Pembimbing Tunggal